Student Name: Ashriel Tayo

Date: 01/30/2026

Project Title: RecipeGenius – Multi-Agent Cooking Assistant

Problem & Motivation

Problem: People waste time searching multiple recipe sites, checking pantry ingredients, adjusting for dietary needs, and creating shopping lists. The process is fragmented across 4-5 different apps/websites.

Target Users:

- Busy home cooks
- College students with limited kitchen equipment
- People with dietary restrictions

Why Agentic AI?

- Single recipes often need adjustments (servings, ingredients, dietary)
- Need to cross-reference pantry items with recipe requirements
- Shopping list generation requires aggregating across multiple recipes

---

Simplified Use Cases

Scenario 1: "Use what I have"

User: "I have chicken, rice, and broccoli. What can I make?"

Agents will: Check pantry, find matching recipes, suggest complete meal.

Scenario 2: "Plan my week"

User: "Give me 3 dinners for 2 people, vegetarian, under 30 minutes"

Agents will: Generate varied recipes, create combined shopping list, adjust servings.

Scenario 3: "Adapt this recipe"

User: Uploads a recipe, says "Make it gluten-free and double the servings"

Agents will: Modify ingredients, adjust quantities, suggest substitutions.

Simple System Design

3-Agent Workflow:

1. **Recipe Finder Agent**
    - Searching recipe databases/APIs
    - Filters by dietary restrictions, time, equipment
    - *Tool:* Spoonacular API (free tier: 150 calls/day)
2. **Pantry Manager Agent**
    - Maintains virtual pantry inventory

    - Checks what ingredients user has vs. needs
    - Suggests substitutions for missing items
    - *Tool:* Local JSON database (no external API needed)
3. **Shopping List Agent**
    - Consolidates ingredients from multiple recipes
    - Groups by grocery store sections (produce, dairy, etc.)
    - Estimates costs if budget provided
    - *Tool:* Simple list generator + export to text/PDF

Frontend: Gradio or Streamlit (simple Python web UI)

APIs: Only Spoonacular API (no complex integrations)

Data: Local SQLite database for user preferences/pantry

Risks & Constraints

Challenges:

- API rate limits (150/day) → Cache results, use efficiently
- Recipe quality varies → Filter by ratings, add "tried and liked" flag
- Ingredient parsing → Use standardized food names